Automating Infrastructure Deployment with AWS CloudFormation

by Mohamad Ikhsan Nurulloh

AWS CloudFormation

Important Disclaimer

- **No Endorsement:** This guide is for educational purposes only and does not imply endorsement by Metabase or AWS.
- **Testing Only:** Intended for testing and development, not production environments.
- **Refer to Official Docs:** Always consult official AWS documentation for production use.
- **Proceed at Your Own Risk:** Use this guide at your own risk and be aware of potential deployment implications.

Overview of AWS CloudFormation

AWS CloudFormation

AWS CloudFormation is a service that lets you define and provision AWS resources using templates, so you can focus on your applications instead of manually managing infrastructure. You write a template describing the resources you need, such as Amazon EC2 instances or Amazon RDS databases, and CloudFormation automatically provisions and configures them in the correct order, handling all dependencies for you.

Business Case for Infrastructure Automation

AWS CloudFormation

AWS CloudFormation lets you define AWS resources as code, enabling fast, consistent, and error-free deployments.

This project uses templates to automate the network layer (VPC and subnet) and the application layer (EC2 instance and security group), ensuring the application setup automatically connects to the network.

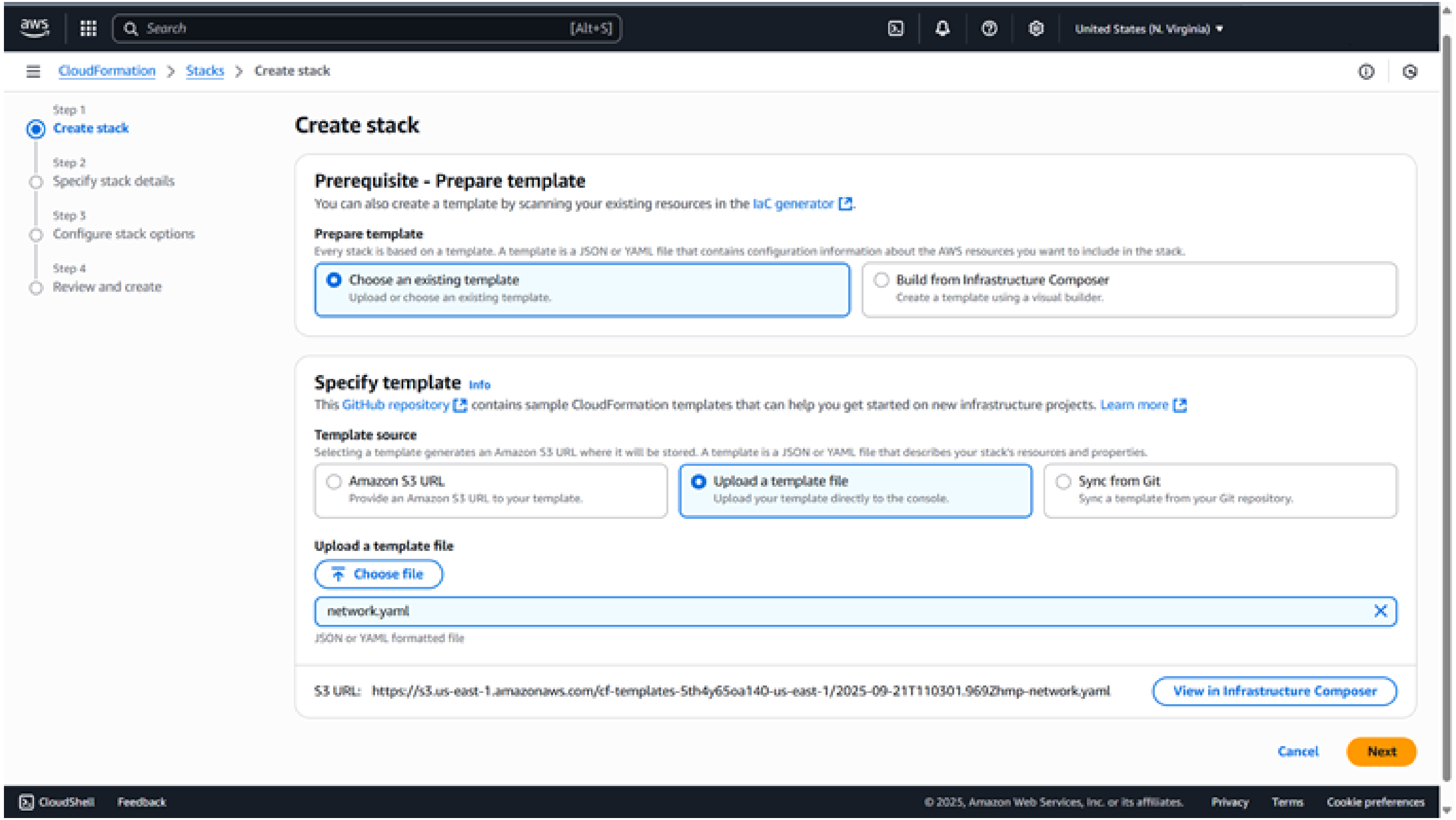
The approach reduces human errors, saves time, and simplifies updates or scaling across different environments like development and productio

Designing the Network YAML Template

```yaml
AWSTemplateFormatVersion: 2010-09-09
Description: >-
  Network Template: Sample template that creates a VPC with DNS and public IPs enabled.

# This template creates:
#   VPC
#   Internet Gateway
#   Public Route Table
#   Public Subnet

######################
# Resources section
######################

Resources:

  ## VPC

  VPC:
    Type: AWS::EC2::VPC
    Properties:
      EnableDnsSupport: true
      EnableDnsHostnames: true
      CidrBlock: 10.0.0.0/16

  ## Internet Gateway

  InternetGateway:
    Type: AWS::EC2::InternetGateway
```

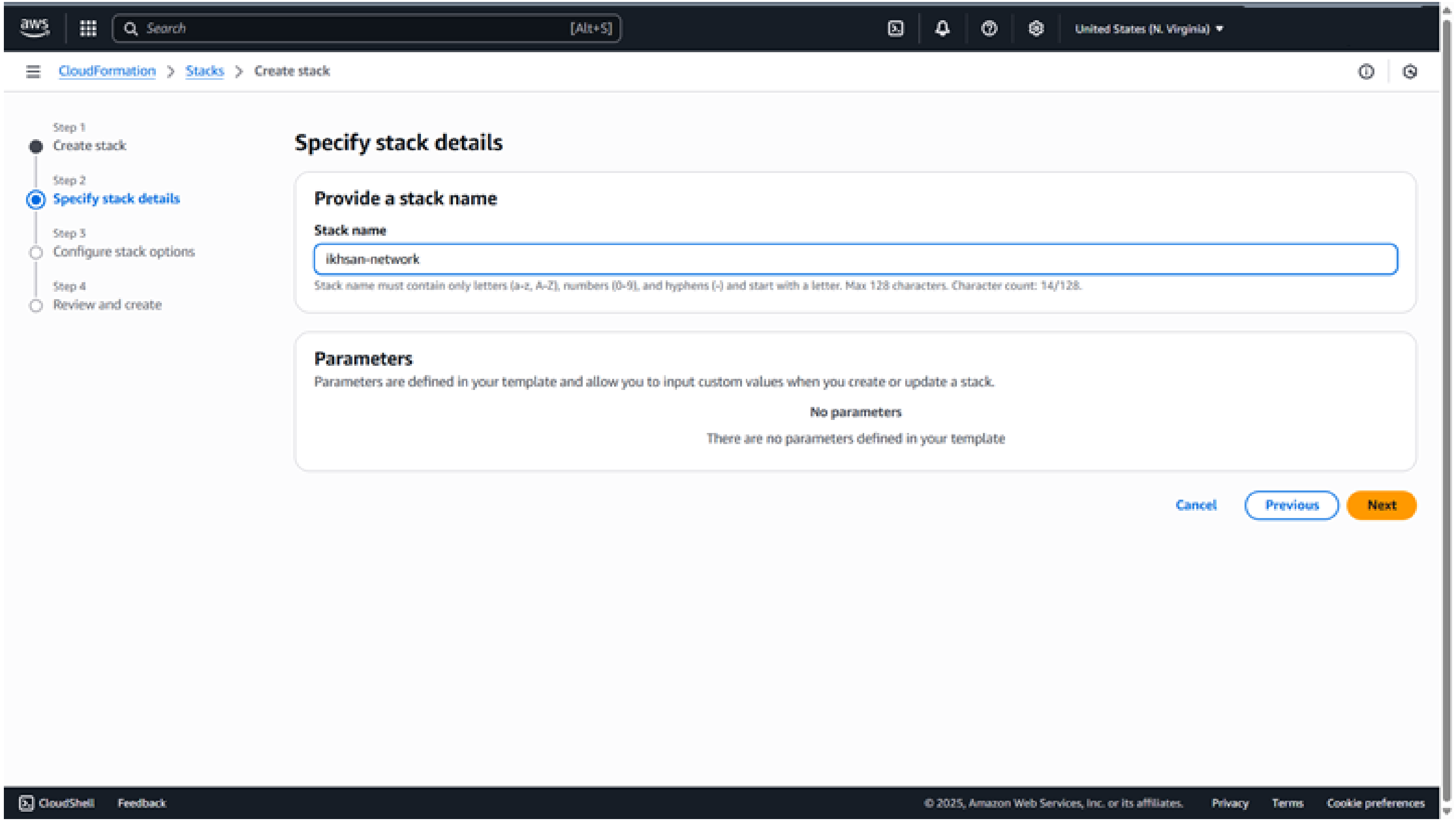
Draft and validate network.yaml to define the VPC, public/private subnets, and related networking resources. Make sure outputs such as VPC ID and Public Subnet IDs are exported for use by other stacks.
For a more complete example, visit this GitHub repo: [https://github.com/ikhsannur1996/cloud-formation].

Initiating the Networking Stack Creation



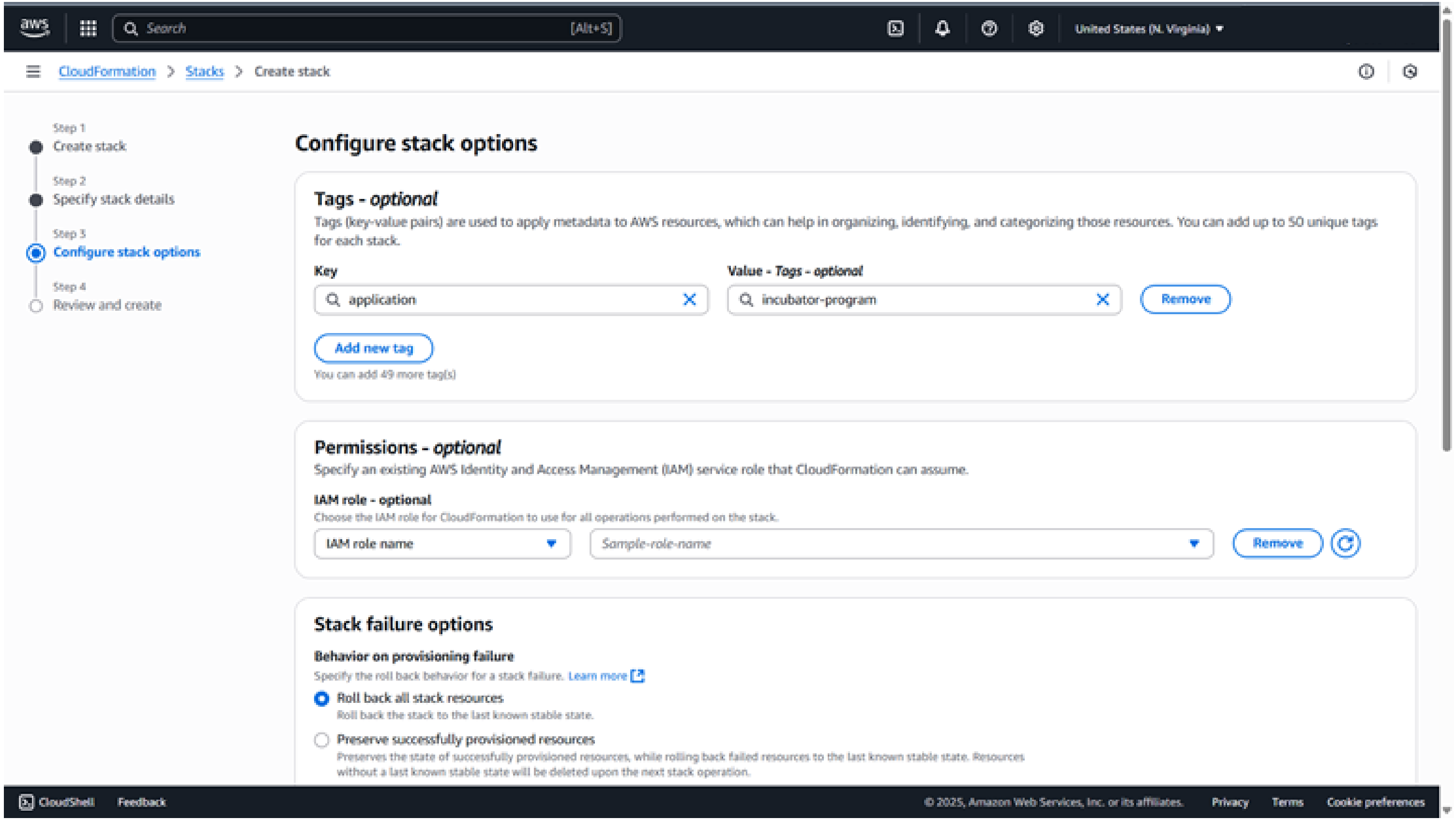
Go to AWS CloudFormation and select Create stack → With new resources (standard).
Upload the template file network.yaml to start creating the networking layer stack.

Defining Networking Stack Parameters



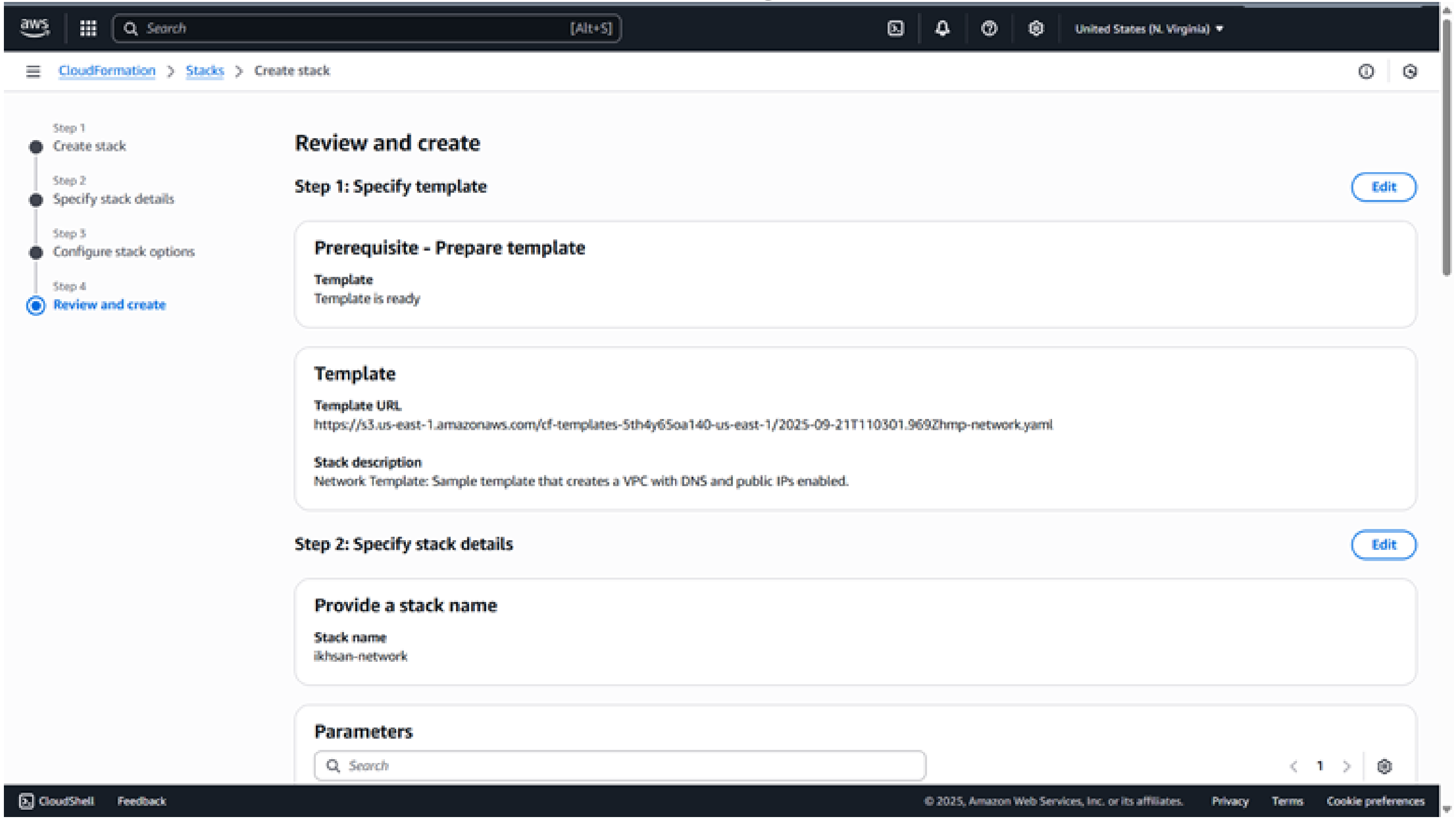
Enter the Stack name as ikhsan-network to identify the networking stack.
No additional parameters are required for this stack.

Configuring Networking Stack Options



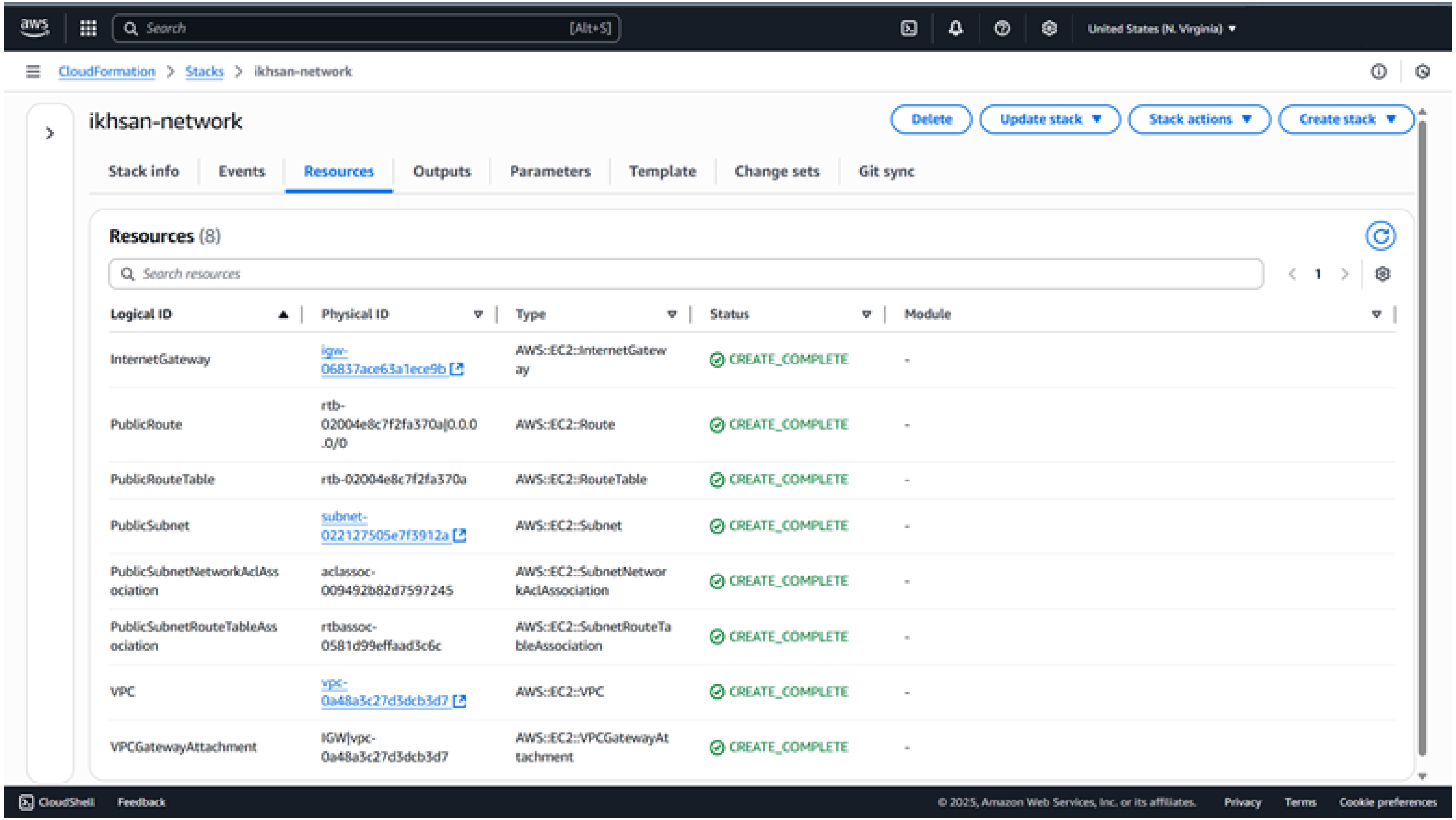
Leave the default settings or add tags and permissions if needed for better resource management.

Review and Launch Networking Stack



Check the template details and configuration, then click Create stack.
Wait until the stack status shows CREATE_COMPLETE, confirming the networking resources have been deployed.

Validating Deployed Network Resources



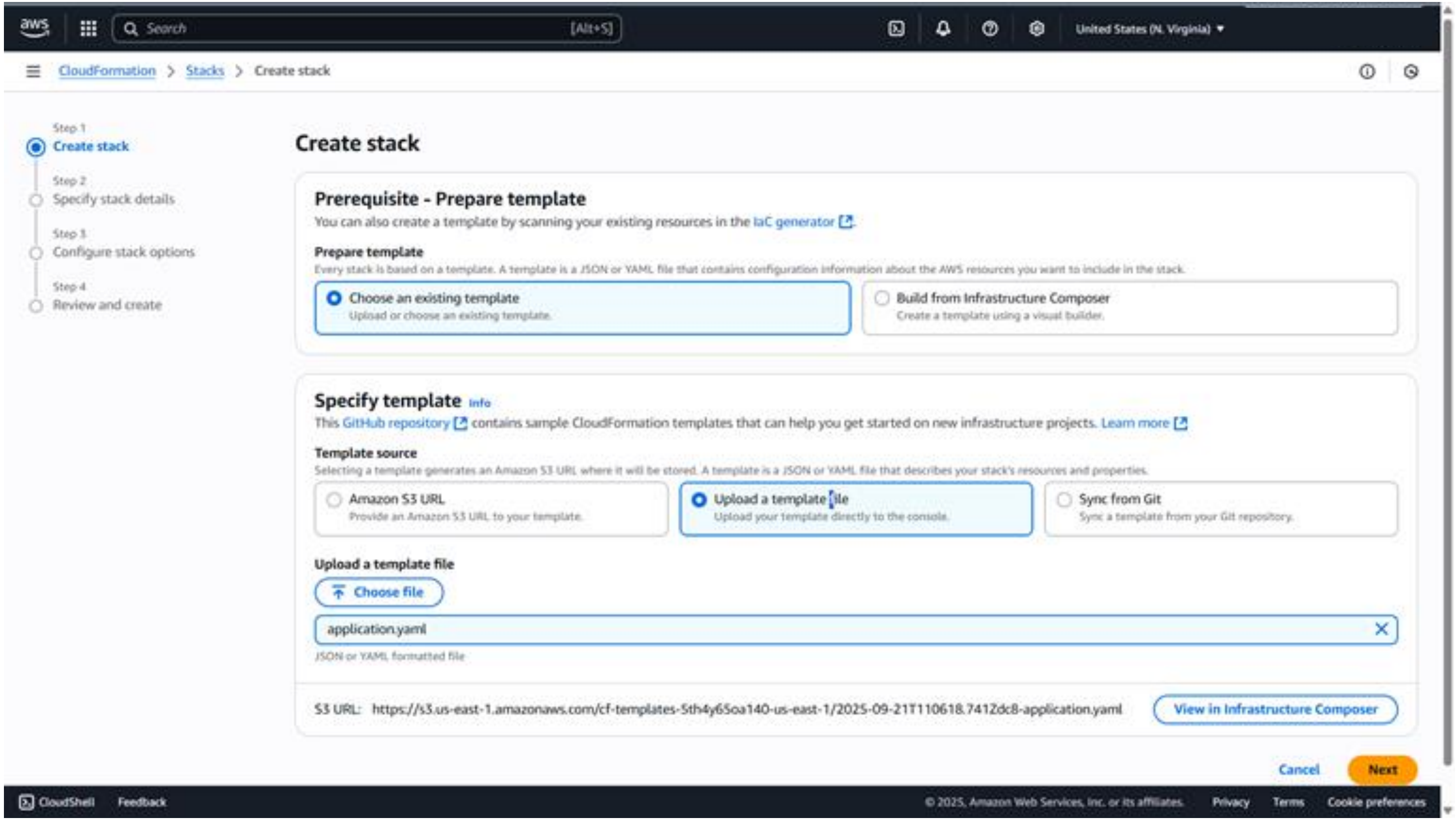
Open the Resources tab to view the created VPC, subnets, and related networking components.
Check the Outputs tab to find the exported VPC ID and Public Subnet IDs for use by other stacks.

Designing the Application YAML Template

```yaml
AWSTemplateFormatVersion: 2010-09-09
Description: >-
  Application Template: Demonstrates how to reference resources from a different stack.
  This template provisions an EC2 instance in a VPC Subnet provisioned in a different stack.

# This template creates:
#   Amazon EC2 instance
#   Security Group

######################
# Parameters section
######################

Parameters:

  NetworkStackName:
    Description: >-
      Name of an active CloudFormation stack that contains the networking
      resources, such as the VPC and subnet that will be used in this stack.
    Type: String
    MinLength: 1
    MaxLength: 255
    AllowedPattern: '^[a-zA-Z][-a-zA-Z0-9]*$'
    Default: ikhsan-network

  AmazonLinuxAMIID:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2
```

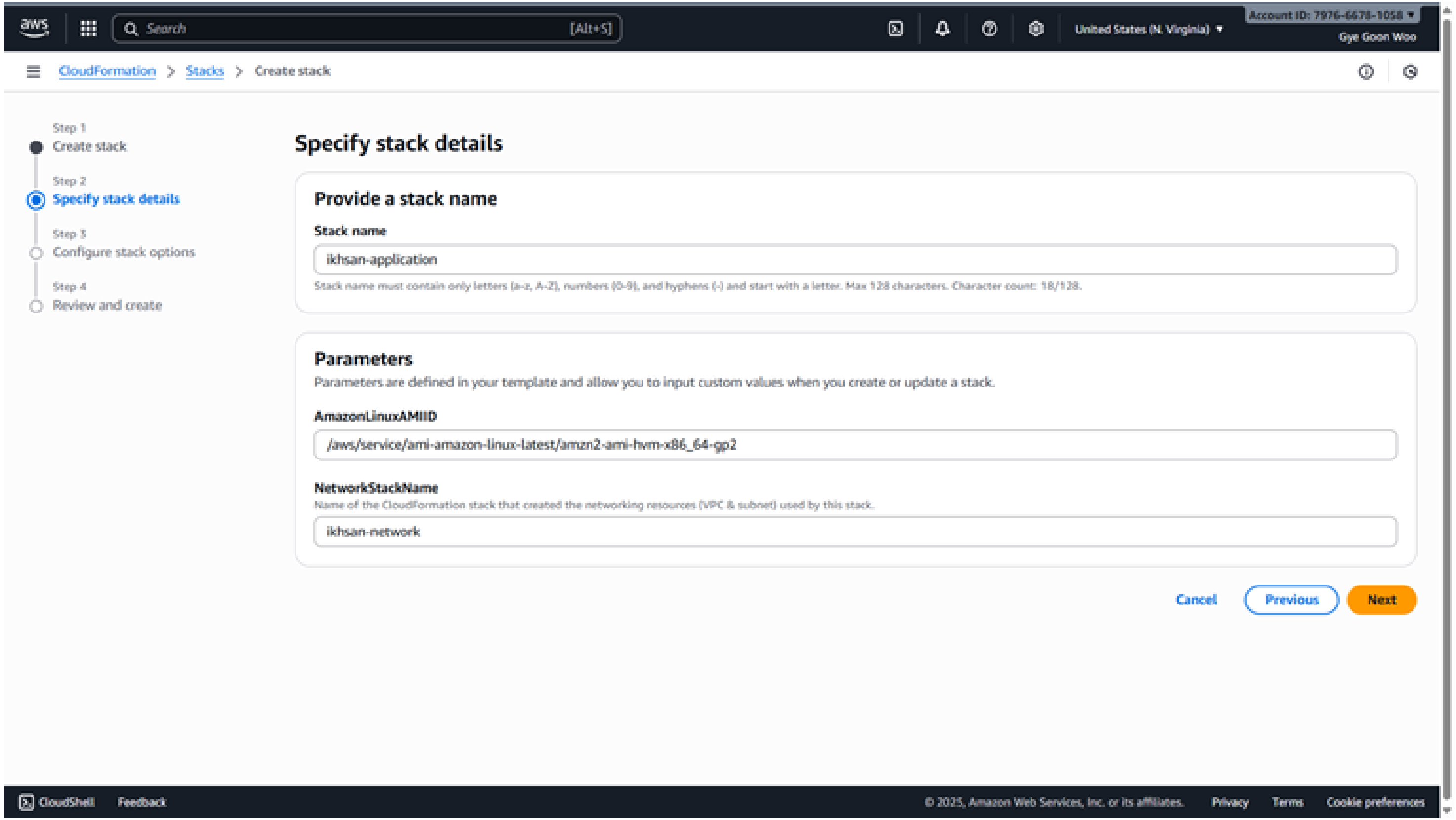
Draft and validate application.yaml to deploy the EC2 instance and security group while referencing outputs from the network stack. Ensure the parameter NetworkStackName is set to import VPC and subnet IDs from the network stack outputs.
For a complete reference setup and template examples, visit this GitHub repo: [https://github.com/ikhsannur1996/cloud-formation].

Launching the Application Stack Creation



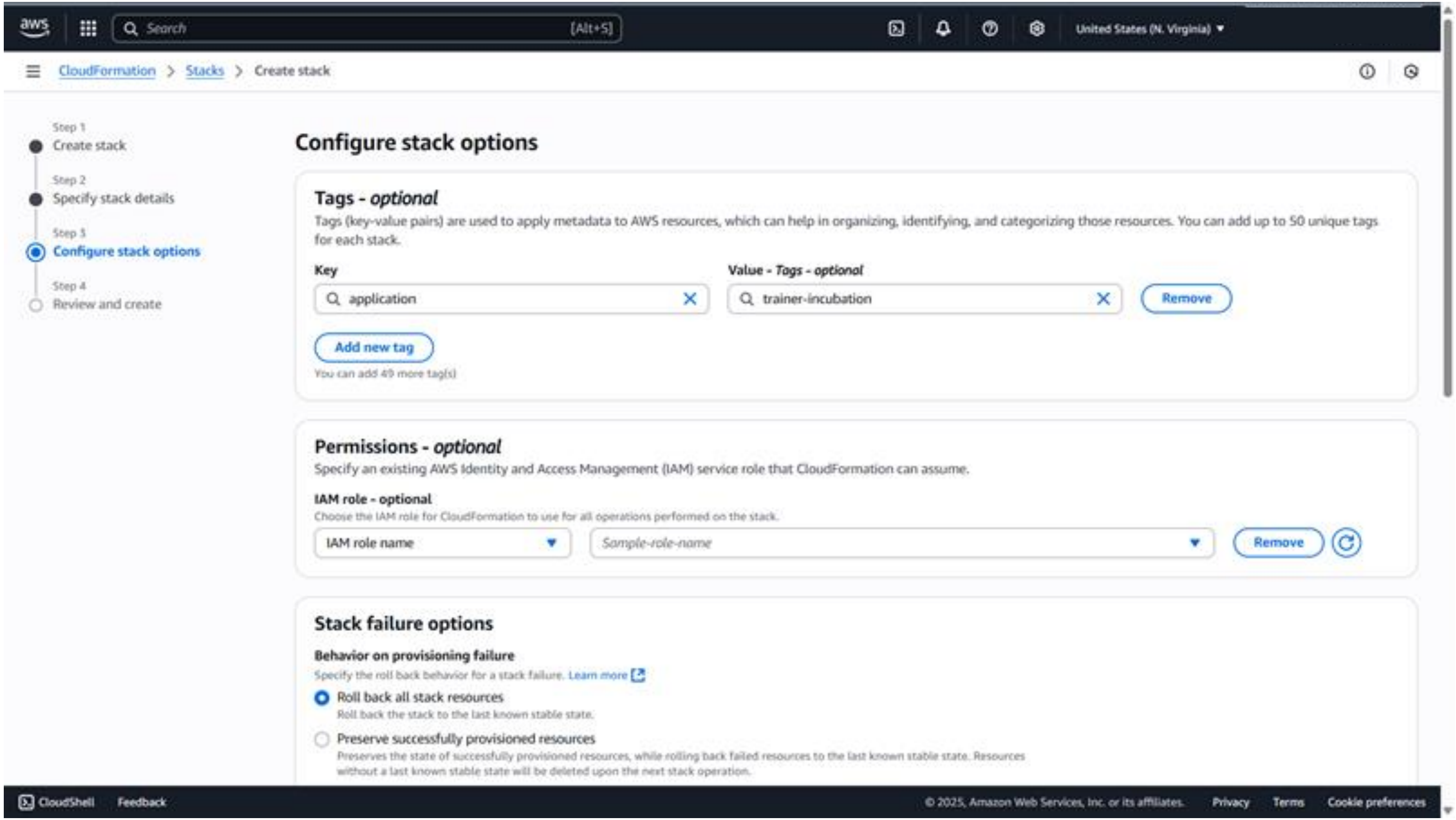
Go to AWS CloudFormation and select Create stack → With new resources (standard).
Upload the template file application.yaml to start creating the application layer stack.

Defining Application Stack Parameters



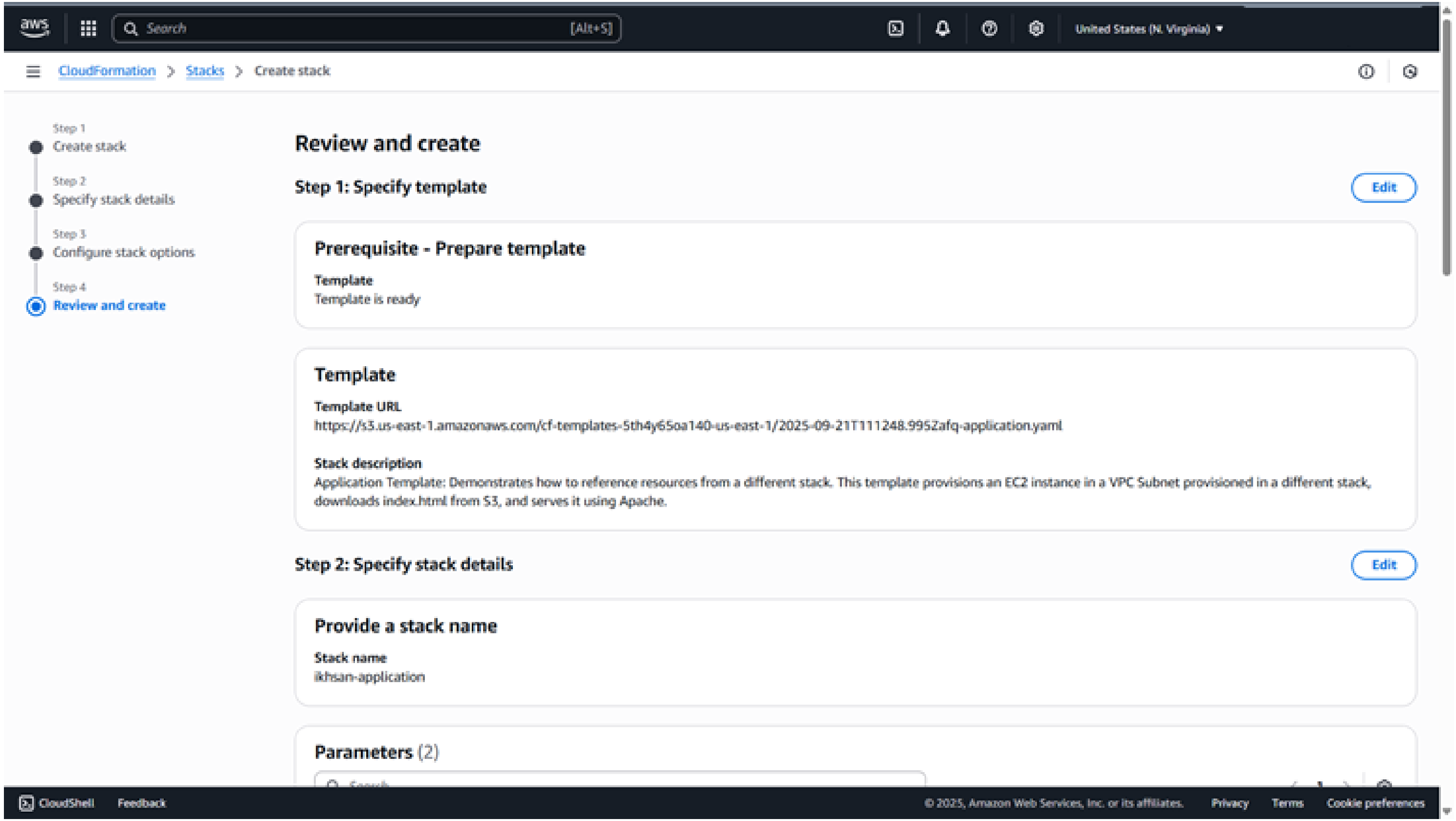
Enter the Stack name as ikhsan-application. Set the parameter NetworkStackName to ikhsan-network so the application stack can import the VPC and subnet IDs from the networking stack outputs.

Configuring Application Stack Options



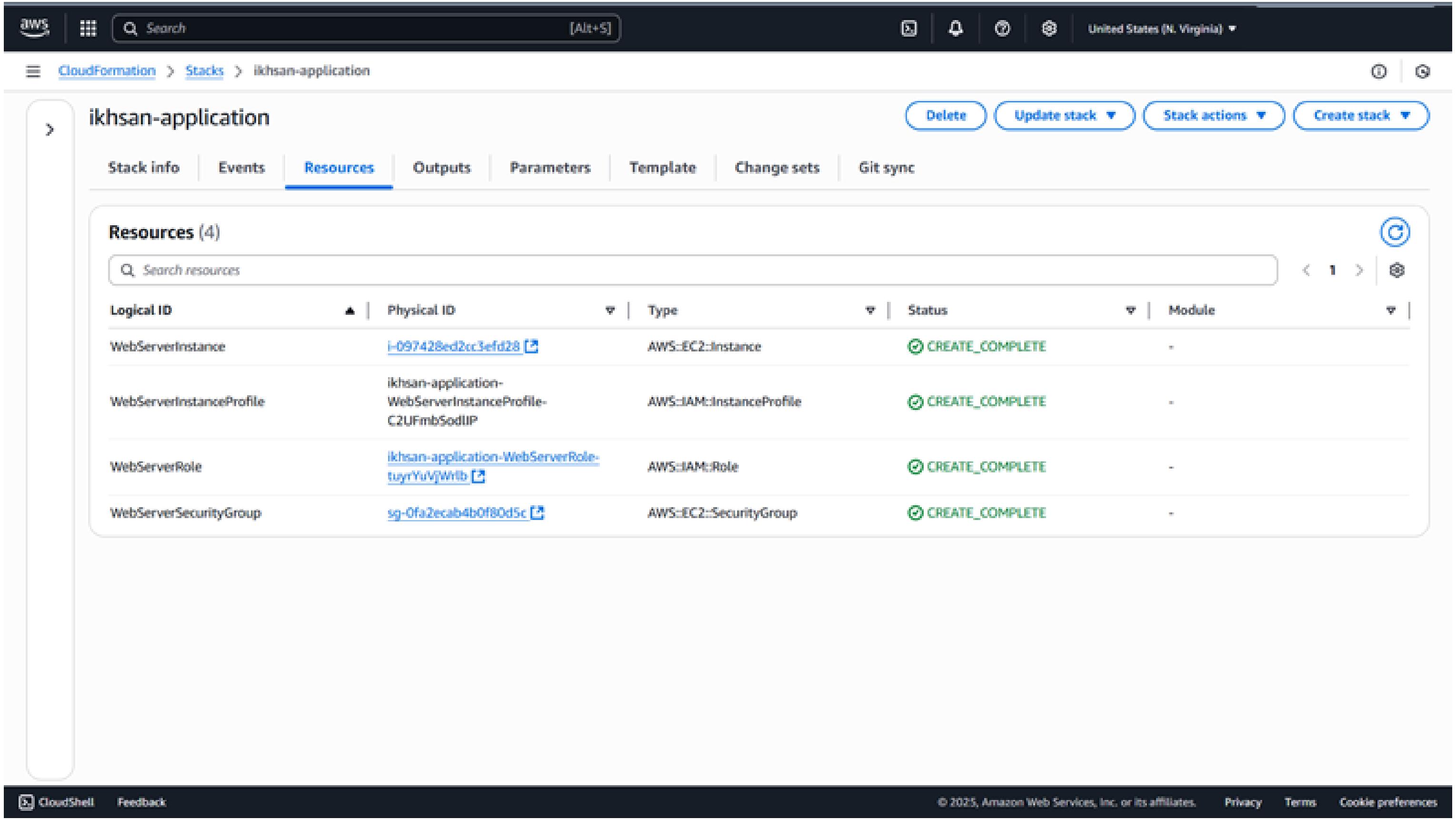
Leave the default settings or add tags and permissions as needed for easy identification and access control.

Review and Deploy Application Stack



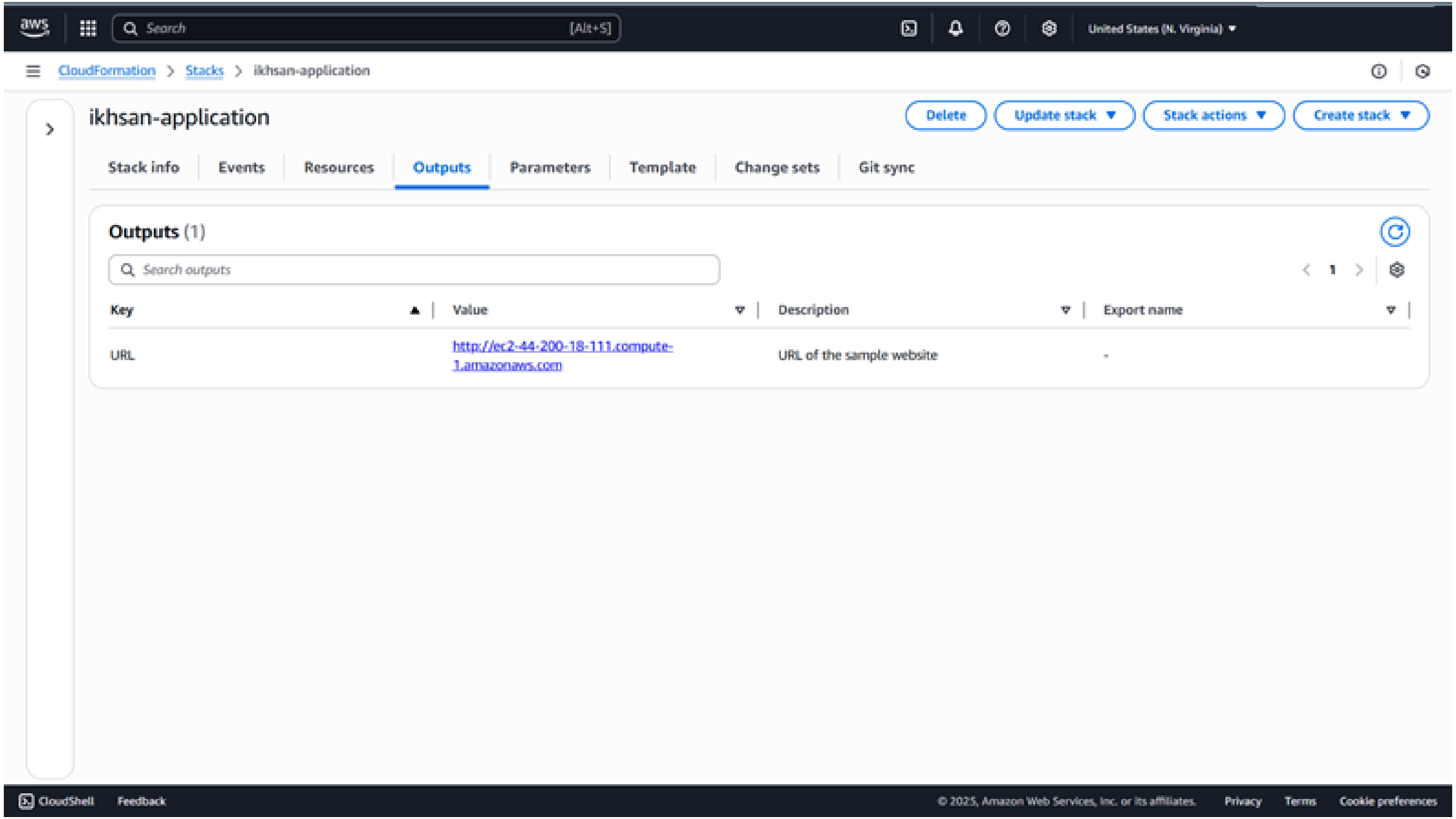
Review the configuration and click Create stack. Wait until the status becomes CREATE_COMPLETE, indicating that the EC2 instance and security group have been successfully deployed.

Verifying Application Resources Deployment



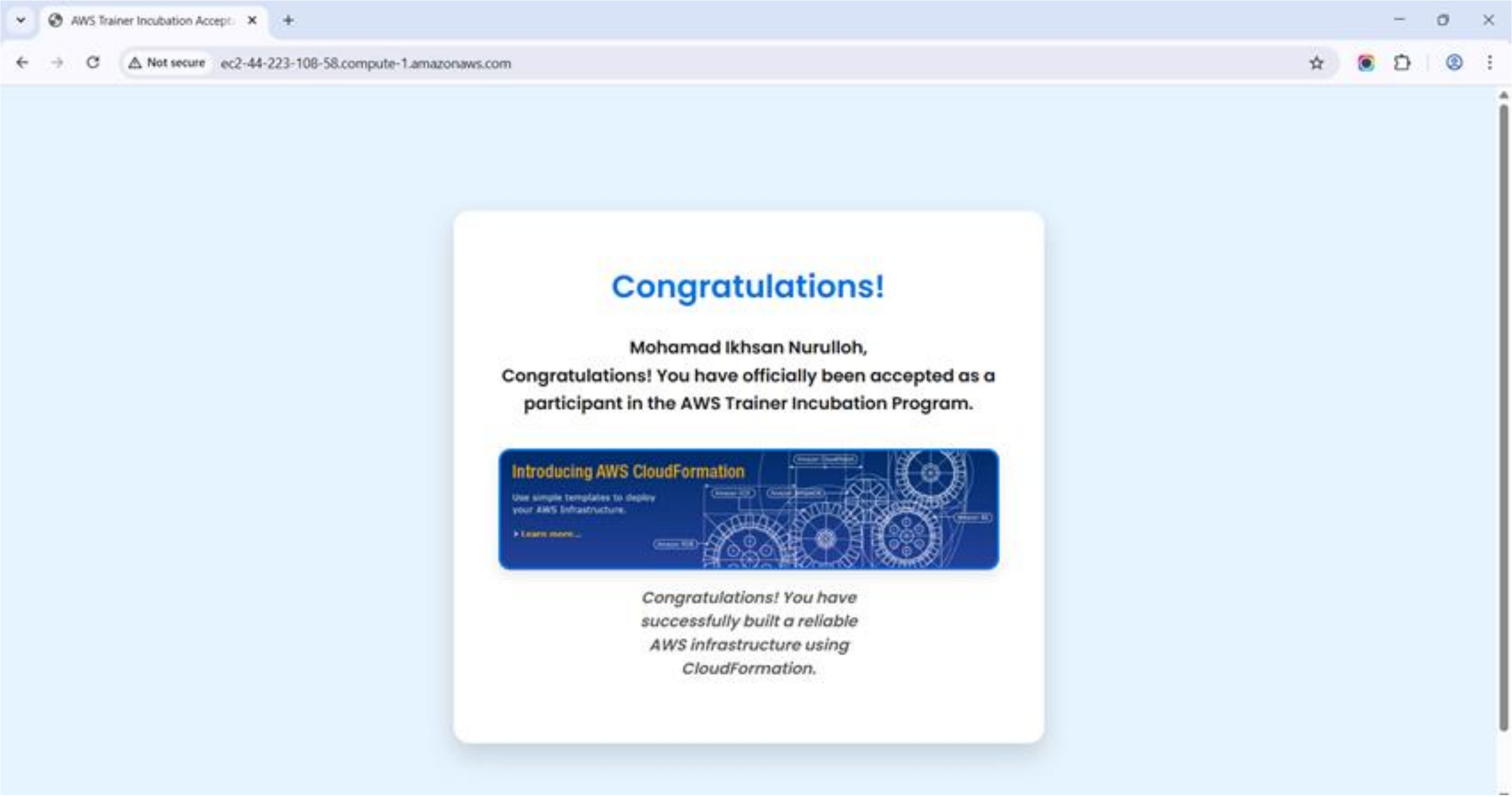
Open the Resources tab to confirm the EC2 instance and security group are successfully created.

Accessing the Application Stack Output



Check the Outputs tab for the application URL, then open it in a browser to verify the application is running on the EC2 instance.

Testing the Deployed Web Application



Open this URL in a web browser to confirm the application is accessible and running correctly on the deployed EC2 instance.

Key Outcomes and Observations

- CloudFormation templates successfully automated the creation of network (ikhsan-network) and application (ikhsan-application) stacks.
- Networking outputs (VPC & Subnet IDs) were reused by the application stack, ensuring clean and consistent infrastructure deployment.
- The application URL in Outputs confirmed the EC2 instance was deployed and running as expected.

Strategic Recommendations for Production

- Store and version CloudFormation templates in Git for easier updates and rollback.
- Integrate with CI/CD pipelines to automate future deployments.
- Add parameters and outputs for flexibility and reusability across environments.
- For production, enhance the template with Auto Scaling, ALB, and CloudWatch monitoring.

Thank You